Information Technology Course
Module Software Engineering
by Damir Dobric / Andreas Pech

ML 24/25-04 Implement the visualization of permanence value

Md Mushfiqul Islam
1438422
islam.md-mushfiqul@stud.fra-uas.de

Proloy Sen
1517624
proloy.sen@stud.fra-uas.de

Rezoyana Islam Bonna
1502329
rezoyana.bonna@stud.fra-uas.de

Abstract - Hierarchical Temporal Memory (HTM) is a biologically inspired machine learning model designed to emulate the neocortex's learning mechanisms based on the sparse distributed representations. This paper proposes an improved visualization approach to visualize permanence value dynamics within the Spatial Pooler, enhancing interpretability and model optimization. By encoding numerical values, we use the HTM Spatial Pooler to generate Sparse Distributed Representations (SDRs) and track changes in permanence over time using Neocortex API's Reconstruct method. Our approach integrates advanced visualization techniques to represent permanence distributions. Through detailed analysis, we explore the impact of encoding choices and noise on permanence evolution. Additionally, we extend our approach by incorporating image data as input to the HTM Spatial Pooler, analyzing its effectiveness in visual pattern recognition and reconstruction. This research enhances the interpretability of HTM networks by providing a clear and insightful visualization of permanence values, enabling better analysis of learning dynamics and synaptic stability.

Keywords - Hierarchical Temporal Memory (HTM), Spatial Pooler, Sparse Distributed Representations (SDRs), Permanence Value Visualization, Input Reconstruction Accuracy

I. INTRODUCTION

Hierarchical Temporal Memory (HTM) is a computational framework inspired by the functioning of the human neocortex. This model is developed by Numenta that specializes in recognizing temporal patterns and modeling data sequences through Sparse Distributed Representations (SDRs). Its architecture emphasizes sparsity and hierarchical processing, enabling effective learning and storage of spatial and temporal relationships.

Neocortex is the brain's center of intelligence and is designed to model complex data sequences. By leveraging temporal hierarchy and sparsity, HTM introduces a groundbreaking technique to machine learning. By drawing parallels to the neocortex, HTM aims to replicate some of the brain's sophisticated learning mechanisms in artificial intelligence.

The Spatial Pooler (SP), a core component of Hierarchical Temporal Memory (HTM), plays a pivotal role in converting raw input data into Sparse Distributed Representations (SDRs). This process mirrors the brain's own mechanism for encoding information, offering more than just a mathematical representation. By utilizing sparsity and distribution, like the neocortex, these SDRs facilitate efficient pattern recognition and information storage.

In this paper, we provide an in-depth analysis of how the Spatial Pooler and Reconstruct() function of "NeocortexApi" work together to achieve accurate input reconstruction, emphasizing the visualization of permanence evolution across different data types. This research enriches our understanding of HTM's reconstructive potential, offering new insights into how encoded representations evolve and are interpreted.

Parallel to this, we explore the visualization of permanence values, a crucial aspect of understanding how the Spatial Pooler functions at a deeper level. Permanence values, which govern the stability of synaptic connections, evolve during learning and can be visualized in various forms.

Furthermore, we examine two distinct representations of permanence values: for image data input and numerical inputs. These visualizations help illustrate how the permanence values change as the input sequences are reconstructed, providing a clearer understanding of the learning dynamics in HTM.

II. LITERATURE REVIEW

Hierarchical Temporal Memory (HTM) is designed to model the structural and functional properties of the neocortex to achieve machine intelligence. Originally introduced by George and Hawkins [1], HTM is grounded in the hierarchical organization of the neocortex, which processes information through a layered structure, enabling the system to learn and recognize complex patterns over time.

HTM's primary strength lies in its ability to learn sequences in an unsupervised manner. It achieves this through the Temporal Memory (TM) algorithm, which allows the system to form temporal dependencies between observed patterns. Hawkins et al. [2] further refined the HTM model by integrating key mechanisms such as sparse distributed representations (SDRs) and cortical learning algorithms, which improve the model's robustness and adaptability in various real-world applications. Their work emphasized HTM's ability to detect anomalies, recognize patterns, and predict future states with minimal labeled data.

The hierarchical nature of HTM enables it to process information at multiple levels, mimicking how the brain processes sensory inputs. It effectively captures spatial and temporal correlations in data, making it highly suitable for tasks that require sequence learning, such as time-series forecasting, anomaly detection, and pattern recognition. HTM's distinct approach to continuous learning, combined with its ability to handle noisy and incomplete data, sets it apart from traditional deep learning models.

SDRs serve as the fundamental representation mechanism in HTM, playing a crucial role in encoding and processing information. Unlike dense representations, SDRs use a sparse and distributed encoding approach, which enhances the model's fault tolerance, generalization capability, and robustness to noise. Ahmad and Hawkins [3] conducted an in-depth analysis of SDRs, demonstrating their mathematical properties, including their ability to represent high-dimensional data efficiently while maintaining semantic similarity.

One of the core benefits of SDRs is their ability to represent overlapping patterns, allowing the system to generalize learned information effectively. This property is particularly beneficial in classification and recognition tasks, where minor variations in input data should not lead to drastic changes in output representations. Pedersen [4] compared SDRs with traditional dense embeddings and found that SDRs provide superior interpretability and resilience against noise, making them more suitable for tasks that require high levels of reliability and robustness.

Moreover, SDRs enable continuous learning by allowing the model to update its knowledge incrementally without catastrophic forgetting, a common issue in artificial neural networks. This feature enhances HTM's adaptability to dynamic environments, making it a viable solution for real-time data processing applications.

Encoders are responsible for converting raw input data into SDRs, ensuring that the system can process diverse data types while preserving semantic meaning. Cui et al. [5] emphasized the significance of encoders in HTM, noting that an effective encoding strategy ensures that similar inputs produce overlapping SDRs, facilitating robust pattern recognition and sequence learning.

HTM utilizes various encoding techniques, including scalar encoding, categorical encoding, and temporal encoding, to handle different data formats. Each encoding method is designed to retain essential input features while minimizing information loss. For instance, scalar encoders are widely used for numerical data, ensuring that nearby values produce highly similar SDRs, which aids in temporal learning.

Yavuz et al. [6] explored optimization techniques for encoders, focusing on improving the efficiency and accuracy of the transformation process. Their research highlighted the importance of parameter tuning in encoders to maximize HTM's learning capabilities while maintaining computational efficiency. Properly designed encoders contribute to HTM's ability to generalize across different input domains, reinforcing its versatility in handling structured and unstructured data.

Input reconstruction refers to the ability of a system to recreate original inputs from their internal representations. In HTM, this process involves reconstructing input data from SDRs, leveraging the sparsity and distributed nature of the representations to achieve accurate reconstructions. Lewis et al. [7] examined HTM's input reconstruction capabilities, demonstrating that the model can effectively reconstruct missing or noisy data, thereby enhancing its resilience in real-world applications.

HTM-based input reconstruction is particularly valuable in applications such as error correction, missing data imputation, and signal restoration. Due to its ability to maintain essential features of input data, HTM can reconstruct meaningful representations even when portions of the input are lost or corrupted. Mhatre et al. [8] applied HTM-based reconstruction techniques to spatial mapping tasks, showcasing the model's ability to learn and recreate spatial environments accurately. Their findings highlight HTM's potential in fields such as robotics and navigation, where spatial awareness and reconstruction capabilities are critical.

Unlike traditional machine learning models that rely on predefined mappings for reconstruction, HTM dynamically learns patterns and adapts its internal representations, making it highly robust in environments where data variability is high. This ability to reconstruct inputs with minimal data loss enhances HTM's applicability in various domains requiring reliable data interpretation.

The integration of HTM, SDRs, encoders, and input reconstruction has led to significant advancements across multiple fields. One of the most prominent applications of HTM is in anomaly detection, where its continuous learning capabilities enable real-time identification of abnormal patterns. Vetrivel et al. [9] applied HTM models to streaming data anomaly detection, demonstrating the system's ability to adapt to changing patterns and detect deviations effectively. Their study highlighted HTM's advantage over traditional anomaly detection techniques, particularly in environments where data distributions change dynamically.

HTM has also been explored in natural language processing (NLP). Dubey et al. [10] investigated the use of HTM in language modeling, leveraging encoders to transform textual data into SDRs. Their findings indicate that HTM can effectively capture linguistic patterns, enabling improved text classification, sentiment analysis, and contextual understanding. Unlike conventional deep learning approaches that require extensive labeled data, HTM's unsupervised learning paradigm allows it to process and analyze text with minimal supervision, making it a cost-effective alternative for NLP applications.

Beyond anomaly detection and NLP, HTM has shown promise in fields such as robotics, cybersecurity, and biomedical signal analysis. Its ability to handle high-dimensional, temporal data makes it an attractive option for industries requiring real-time decision-making and adaptive learning. Furthermore, ongoing research continues to refine HTM's algorithms, focusing on enhancing its computational efficiency and scalability for large-scale implementations.

III. METHODOLOGY

Our approach focuses on accurately reconstructing original input data using the Hierarchical Temporal Memory (HTM) framework and the Neocortex API's Reconstruct() method.

Initially, diverse input data types, including numerical values (0–99) and images, are processed. Image data is first binarized using an Image Binarizer, converting them into binary representations.

Next, all input data undergo transformation into int[] arrays consisting solely of 0s and 1s. These binary-encoded arrays serve as the input for our experiment. The HTM Spatial Pooler is then employed to convert these encoded arrays into Sparse Distributed Representations (SDRs), which capture the essential features of the input in a sparse and distributed manner.

Once the SDRs are generated, the reconstruction process begins. Utilizing the Neocortex API's Reconstruct() method, the original encoded representations are reconstructed based on the permanence values derived from the reconstruction process. This step aims to restore the structure of the original int[] arrays with high fidelity.

To evaluate reconstruction accuracy, we employ two primary visualizations: heatmaps and int[] sequence comparisons. Additionally, a similarity check is performed between the original and reconstructed input arrays, measuring the effectiveness of the HTM algorithm in preserving the structural integrity of the input data.

The ultimate goal of our methodology is to assess the HTM framework's capability in reconstructing inputs using the Neocortex API's Reconstruct() method, ensuring the faithful recreation of the original encoded representations. (Figure1)

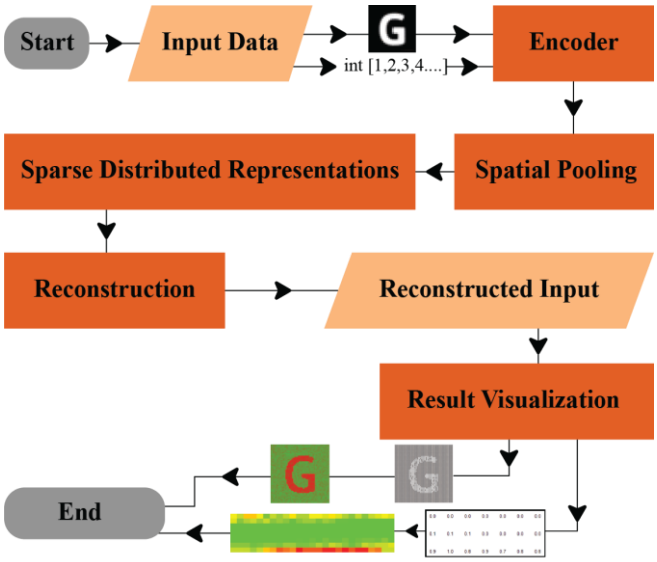

Figure 1: Visual representation of the whole experiment

A. *Types of Data Used in the Experiment:*

1) Numerical Data: A scaler encoder is applied to numerical values between 0 and 99, converting each value into a 200-bit encoded form. These encoded forms are subsequently converted into integer arrays (int[]), where each array corresponds to a specific numerical input. (Figure2)

Figure 2: Encoded Numerical Inputs

2) Images Data: For image data, an Image Binarizer is employed to transform the images into binary formats. The size of the resulting encoded arrays corresponds to the original image dimensions. For example, a 28x28 pixel image produces a 784-element array, with each pixel represented by a binary value. (Figure 3)

Figure 3: Encoded image inputs

The encoded arrays of both types are used as input data in our experiment, with the goal of accurately reconstructing the original data using the Hierarchical Temporal Memory (HTM) framework.

B. *Reconstruction Method Workflow*

In the Hierarchical Temporal Memory (HTM) framework, the Reconstruct method plays a crucial role in reversing the transformation of input data into Sparse Distributed Representations (SDRs) by the Spatial Pooler. This method aims to approximate the original input from the activated SDRs, allowing for a deeper understanding of how information is encoded and preserved within the HTM system.

The Reconstruct method works by leveraging the learned connections and activation patterns of the Spatial Pooler to estimate the most probable input that led to a given SDR. It effectively performs a mapping from the sparse high-dimensional representation back to a more human-interpretable form, making it an essential tool for debugging, visualization, and validation of HTM models. To enhance the accuracy and reliability of reconstruction, we introduced several refinements. The general workflow of the reconstruction method is depicted in Figure 4, outlining its key steps and processes:

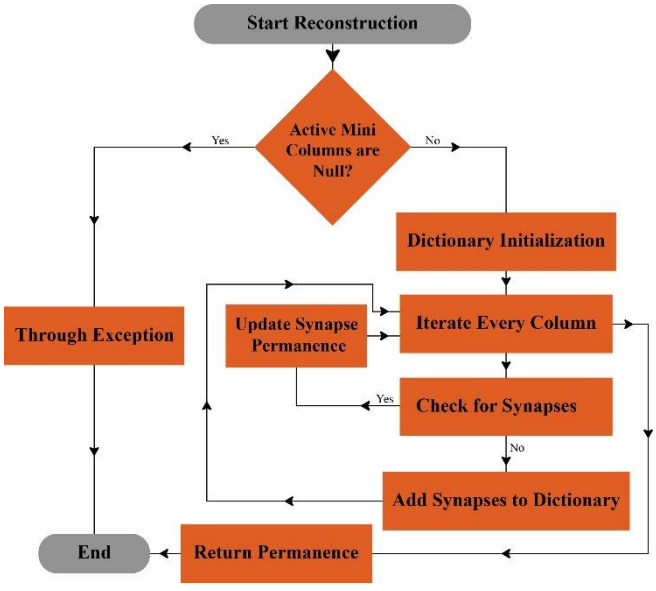

Figure 4: Workflow of the Reconstruction Method

*1) Validation***:** The method starts by performing a comprehensive validation, immediately throwing an ArgumentNullException if the input array of active mini-columns is null. If the array passes this check, the process continues seamlessly, proceeding to complete the Reconstruction operation.

2) Column Retrieval: This step involves retrieving the list of columns associated with the active mini-columns from the connections. The activeMiniColumns parameter in the Reconstruct method is an integer array representing the indices of the mini-columns considered active.

A mini-column refers to a group of cortical neurons. In algorithms like hierarchical temporal memory (HTM), when input patterns match specific criteria, such as overlapping patterns or sensory stimuli, mini-columns are activated.

The current approach uses the activeMiniColumns to identify the active mini-columns. From there, it reconstructs a dictionary that holds the cumulative permanence values of the synapses connected to these active mini-columns. These permanence values play a crucial role in determining the connection strength between neurons in the cortical region.

3) Key components for the process: In the reconstruction process, two key components proximal dendrites and synapses play crucial roles. Proximal dendrites are part of each column in the cortical model, acting as the primary interface for receiving input from neighboring columns or directly from the input data. They are responsible for detecting spatial patterns in the input data and facilitating the spatial pooling process. Synapses, on the other hand, are the connections between the proximal dendrites and the input space or other columns within the network. Each synapse has an associated permanence value, which indicates the strength of its connection.

During reconstruction, the permanence values of these synapses are accumulated in a dictionary, where each input index is mapped to its associated permanence. This process updates existing dictionary entries or adds new ones as needed. The proximal dendrites are essential for determining which synapses are active, and the synapses themselves contribute to the overall strength of the connections. The synaptic permanence values help determine how effectively a column will activate in response to input. Thus, both components are integral to the reconstruction method, as they work together to map input data to the network's synaptic structure.

4) Process of reconstruction: The reconstruction process begins by iterating through each active mini-column, accessing the synapses connected to its proximal dendrite. The proximal dendrites capture input from neighboring columns or directly from the data, playing a crucial role in identifying spatial patterns. Synapses connect these dendrites to input data or other columns, with each synapse having a permanence value that determines the strength of the connection. During the reconstruction, the permanence values for each synapse are accumulated in a dictionary based on the input index, updating existing entries or adding new ones as needed. The process concludes with the return of a reconstructed dictionary, which maps each input index to its associated permanence value.

C. Proposed Method for final visualization:

Our approach to visualizing permanence values and assessing similarity for numerical and image inputs revolves around leveraging the Reconstruct method in the Spatial Pooler (SP). This technique enables us to retrieve permanence values associated with active columns that emerge during the learning phase. The following methodology ensures a structured and comprehensive analysis of spatial patterns in both numerical and image-based data.

1) Numerical Input Processing: To begin, each numerical value is encoded into a Sparse Distributed Representation (SDR) using an appropriate encoder, ensuring that essential features are well captured. The SDR is then passed through the spatial pooler, where active columns are determined without triggering learning, preserving the integrity of the original input. Using the Reconstruct method in the SP, we extract permanence values from these active columns and store them in a structured dictionary called allPermanenceDictionary, which systematically organizes all potential input indices. To provide a complete representation, permanence values for inactive columns—which do not contribute directly to the SDR—are assigned a default value of 0.0, ensuring that all possible input bits are accounted for.

To standardize the permanence values for better interpretability, we apply normalization and thresholding, refining them into a structured format. These values are then converted into a list of integers to facilitate downstream analysis. To measure the effectiveness of the reconstruction process, we compute Jaccard similarity, quantifying the overlap between the original encoded SDRs and their reconstructed permanence values. Finally, we employ heatmaps to visually represent spatial patterns embedded within the HTM network. Additionally, similarity plots are generated to illustrate the degree of alignment between the original SDRs and reconstructed permanence distributions, enhancing interpretability.

2) Image Input Processing: For image-based data, the process follows a similar pipeline but is adapted to suit the complexity of visual features. The workflow commences with reading and binarizing image files from a designated dataset, converting them into a format suitable for spatial pooling. Once binarized, each image undergoes spatial pooling, wherein active columns corresponding to significant patterns in the image are identified. The permanence values for these active columns are then reconstructed and systematically stored in the allPermanenceDictionary, ensuring a complete representation of the input space.

As in the numerical case, normalization and thresholding are applied to standardize the reconstructed permanence values. The fidelity of this reconstruction is again assessed using Jaccard similarity, allowing us to measure how closely the reconstructed values align with the original binarized inputs. The final stage of the process involves generating heatmaps that provide an intuitive visualization of the learned spatial patterns. Furthermore, similarity plots are constructed to reveal how accurately the reconstructed permanence values capture the structural properties of the original image data.

To refine our analysis, we introduce dynamic thresholding techniques to adaptively set thresholds based on data distribution, ensuring robustness across varied datasets. Additionally, statistical matrics beyond Jaccard similarity, such as cosine similarity or entropy-based measures, could be explored to offer a more nuanced evaluation of reconstruction fidelity. The visualization pipeline can further benefit from interactive heatmaps, enabling real-time exploration of permanence patterns across different input types.

IV. IMPLEMENTATION

Our implementation focuses on visualizing permanence value dynamics within the Hierarchical Temporal Memory (HTM) Spatial Pooler to enhance interpretability and model optimization. The process begins with data encoding and SDR generation, where numerical values are transformed into Sparse Distributed Representations (SDRs) using appropriate encoding techniques. Additionally, image data is incorporated through custom encoding methods, enabling the Spatial Pooler to analyze and recognize visual patterns. The HTM Spatial Pooler then processes these SDRs, producing sparse activations based on input patterns.

To track permanence value dynamics, we utilize the Neocortex API's Reconstruct method, extracting and monitoring permanence changes over time. This enables us to assess stability and adaptability under different encoding schemes. Advanced visualization techniques, such as heatmaps, thresholding methods, and graphical representations, are employed to illustrate permanence distributions and uncover meaningful patterns within the network. Through comparative analysis, we evaluate how permanence values evolve across numerical and image-based inputs, refining our understanding of HTM's learning dynamics. These insights provide a deeper understanding of how HTM captures and retains information over time, adapting to various data types. Ultimately, our approach bridges the gap between raw data representation and intelligent pattern recognition, improving the interpretability of HTM's learning mechanisms.

A. Permanence Value Normalization

To ensure the permanence values derived from the Spatial Pooler in the Hierarchical Temporal Memory (HTM) framework are both consistent and comparable, we introduced a thresholding function for normalization. This tep is crucial for streamlining the visualization and analysis, as it effectively transforms the continuous permanence values into a binary format, which simplifies the interpretation process.

Choosing the right threshold value was fundamental to achieving the desired normalization. After extensive experimentation and iterative adjustments, we determined the optimal threshold values for different types of data. For numerical inputs, the threshold was set at 8.3, while for image data, we arrived at a value of 69 These thresholds were selected based on thorough testing, ensuring the normalized permanence values closely align with the original data. By carefully altering the threshold values and assessing their impact on the output, we were able to refine the normalization process, balancing the preservation of input characteristics with meaningful analysis.

This methodical approach underscores the importance of precise parameter selection when visualizing and interpreting permanence values within the HTM framework. Ultimately, it deepens our understanding of hierarchical temporal processing, both for numerical data and image data sets.

B. Matrics Visualization

After normalizing the permanence values, we represent them in a structured matrics format to facilitate an intuitive understanding of how permanence values are distributed across different input indices. This matrics-based visualization provides a spatial representation of the reconstructed permanence values, offering deeper insights into how the HTM Spatial Pooler processes and retains input information.
Each row and column of the matrics corresponds to specific input indices, where the values represent the normalized permanence scores. For numerical data, the matrics captures how permanence values are distributed across different encoded bit positions, while for image inputs, the matrics structure aligns with the spatial dimensions of the original image, preserving the visual context of the input.

C. Heatmap Visualization

To effectively interpret the reconstructed permanence values, we generate heatmaps from the structured permanence matrics. These heatmaps serve as a visual indicator of the permanence values associated with each input index, providing insights into the strength of synaptic connections within the Hierarchical Temporal Memory (HTM) network. The concept of "heat" in the heatmap corresponds directly to the permanence value of each synapse, where warmer colors indicate stronger and more stable connections, while cooler colors represent weaker or less significant connections. This gradient representation allows us to intuitively assess how permanence values evolve over different input indices.

In the context of HTM, synapses act as connections between the proximal dendrites of columns and the input data or other columns. Each synapse has an assigned permanence value, determining the likelihood of its activation. By visualizing these permanence distributions in a heatmap, we

gain a deeper understanding of how the network processes both numerical and image inputs. The "DrawHeatmaps" function plays a crucial role in this visualization process, seamlessly integrating heatmap data with normalized permanence values and original encoded inputs. This function ensures that the reconstructed permanence values are mapped effectively, allowing for an insightful comparative analysis between the original encoded inputs and the reconstructed outputs.

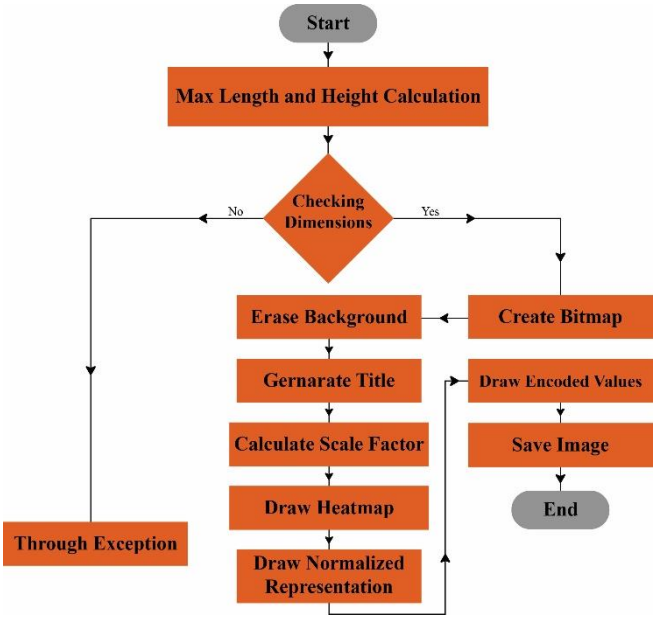

Figure 5: Visual representation of Draw Heatmap

For numerical inputs, the heatmap representation reveals how permanence evolves for different encoded bit positions over time, making it easier to assess reconstruction fidelity. For image inputs, the heatmap aligns with the spatial dimensions of the original image, preserving structural details and enabling a clear comparison between reconstructed and original patterns.

D. Input and output comparison

By leveraging matrics visualization, we enable a clearer comparative analysis between original encoded inputs and their reconstructed counterparts. This not only aids in evaluating the efficiency of the HTM model but also provides a structured representation that enhances the interpretability of permanence evolution across different input types. To evaluate the accuracy of the reconstructed inputs, we compare the original encoded inputs with the reconstructed outputs using the Jaccard Similarity Coefficient (JSC), a widely used metric for measuring the similarity between two binary sets. This method is particularly well-suited for our experiment, where inputs and outputs are represented as binary arrays consisting of 0s and 1s. The Jaccard Similarity Coefficient is calculated as the ratio of the intersection of active bits in both the original encoded input and the reconstructed output to their union. Mathematically, it is defined as:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

Where A represents the set of active bits in the original encoded input, B represents the active bits in the reconstructed output. |A∩B| denotes the number of common active bits, and |A∪B| denotes the total number of unique active bits in both sets. A Jaccard similarity score closer to 1 indicates a high degree of similarity, meaning the reconstruction retains most of the original input structure, whereas a lower score signifies a significant deviation between the reconstructed and original input. For example, given two binary input arrays:

A = [1,0,1,1,0,1,0,0,1,0]
B = [1,1,1,0,0,1,0,1,1,0]

We calculate:

$$J(A,B) = \frac{4}{4+1+2} = \frac{4}{7} \approx 0.571$$

This result indicates that 57.1% of the active bits in the original input are retained in the reconstructed output, demonstrating a relatively high reconstruction fidelity. By computing Jaccard similarity scores across various numerical and image inputs, we assess the effectiveness of our HTM-based reconstruction model. Additionally, visualizing these similarity scores through bar graphs provides an intuitive representation of the reconstruction accuracy across different data points.

E. Visualization of the comparison

The visualization of the comparison between original encoded inputs and reconstructed outputs plays a crucial role in assessing the effectiveness of the reconstruction process. By plotting the similarity scores, we gain valuable insights into the performance of the HTM network in preserving structural integrity during reconstruction. This function generates a bar graph where each bar represents an individual input, and its height corresponds to the Jaccard similarity score between the original and reconstructed versions. Higher bars indicate stronger similarity, implying that the reconstruction successfully retains most of the original input features, whereas shorter bars highlight discrepancies in the reconstruction process. To ensure clarity and interpretability, the function scales the bar heights based on the maximum similarity value in the dataset, providing a clear comparative representation across different inputs. Additionally, axis labels, a title, and a proper scale are integrated into the visualization, making it easier to analyze patterns, detect inconsistencies, and identify potential areas for improvement. This graphical representation serves as an intuitive tool for understanding the distribution of similarity scores across various numerical and image-based inputs, ultimately enabling a deeper evaluation of the HTM framework's reconstruction performance.

V. RESULT ANALYSIS

In this section, we present the results of our study, focusing on the visualization and evaluation of permanence value dynamics in the HTM Spatial Pooler. Our analysis is divided into two main parts: numerical input reconstruction and image input reconstruction. By leveraging heatmap visualizations, we illustrate how permanence values evolve during the learning process and assess the effectiveness of the Spatial Pooler in reconstructing different types of inputs.

1) For numerical inputs: For numerical inputs, the reconstructed permanence values are represented both as a structured matrics and a corresponding heatmap to provide a comprehensive visualization of the reconstruction accuracy. The matrics representation offers a direct numerical insight into how permanence values are distributed across input indices, highlighting the structural patterns captured by the HTM network. Each value in the matrics corresponds to a specific permanence score, reflecting the network's learned representation (Figure 6). Complementing this, the heatmap transforms these numerical values into a visually intuitive format using a color gradient, where warmer colors indicate stronger permanence values and cooler colors represent weaker or absent connections (Figure 7).

0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.1	0.1	0.1	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.1
0.8	0.8	0.8	0.9	1.0	0.8	0.9	0.7	0.8	0.8	0.7	0.7	0.0	0.0

Figure 6: Output heatmap matrics

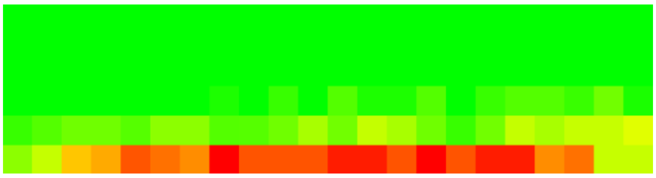

Figure 7: Output heatmap for numerical input.

2) For image inputs: For image inputs, the reconstructed permanence values are visualized through a matrics(Figure 8), after that we have generated a heatmap from it, where warmer colors indicate stronger connections and cooler colors represent weaker ones (Figure 9). This heatmap helps assess how accurately the HTM network reconstructs the original binarized image by comparing the reconstructed permanence values with the original encoded input.

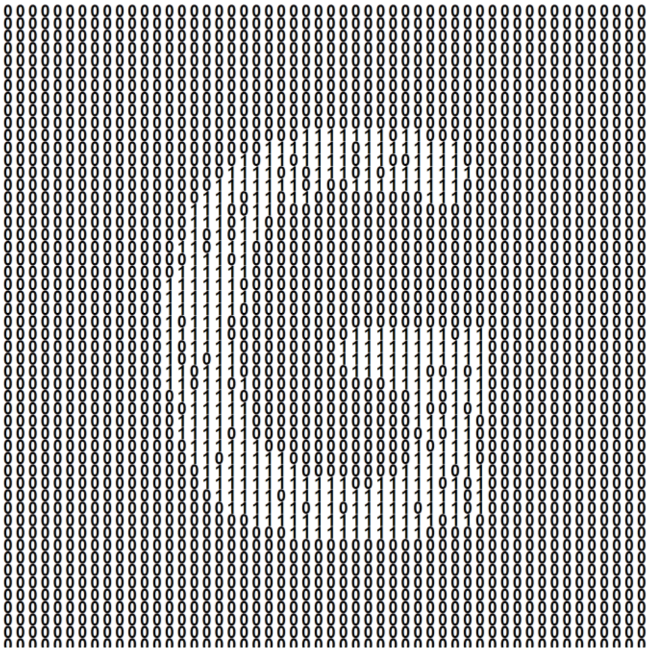

Figure 8: Reconstructed matrics for image input.

Figure 9: Output heatmap for image input.

To further validate the accuracy of our input reconstruction, we analyze the similarity between the original encoded inputs and the reconstructed inputs using the Jaccard similarity coefficient. This metric provides a quantitative measure of how closely the reconstructed inputs align with the original encoded representations, offering deeper insights into the effectiveness of the HTM Spatial Pooler.

In this section, we present the similarity analysis results for both numerical and image inputs. For numerical inputs, we compute the Jaccard similarity for a range of integer values (0 to 99), illustrating the consistency of the Spatial Pooler in preserving the structural integrity of the encoded representations. Corresponding figures (10, 11, 12, 13, and 14) showcase the similarity trends across different numerical inputs.

For image inputs, we extend our analysis to visual data by examining the similarity between the original and reconstructed image patches. Given the high-dimensional nature of image encoding, the Jaccard similarity provides a valuable measure to assess how well the Spatial Pooler retains the core structural features of visual inputs.

By presenting similarity bar graphs for both input types, we offer a comparative analysis of the reconstruction accuracy, highlighting the strengths and potential limitations of the HTM model in handling different data modalities. These results contribute to a better understanding of how HTM networks process, learn, and reconstruct diverse input types.

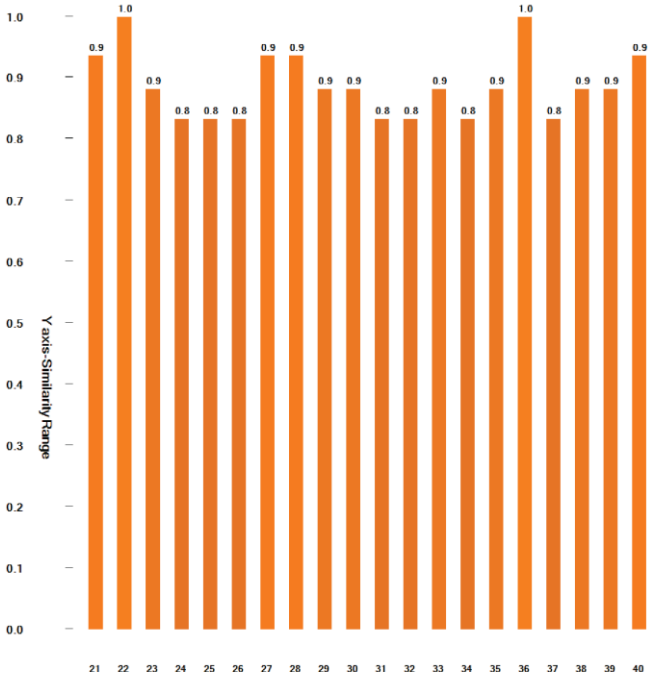

Figure 11: Similarity graph for input 21 to 40

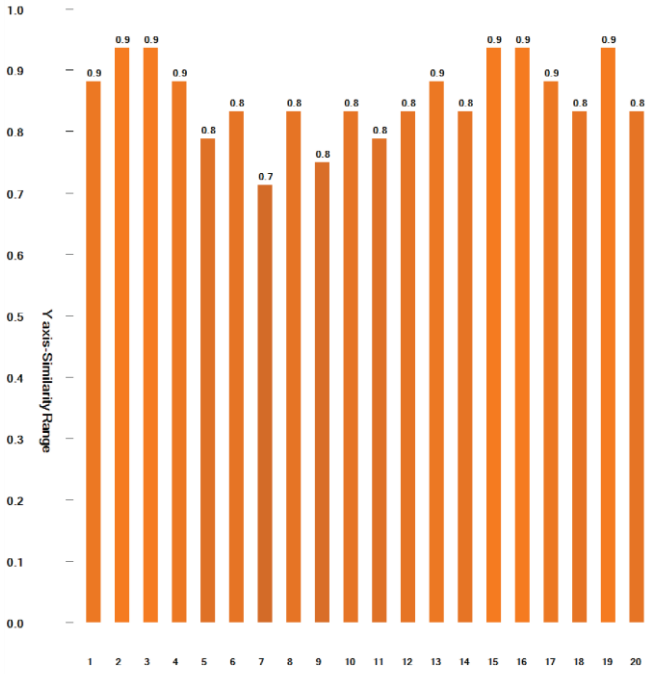

Figure 10: The similarity graph of input 1 to 20

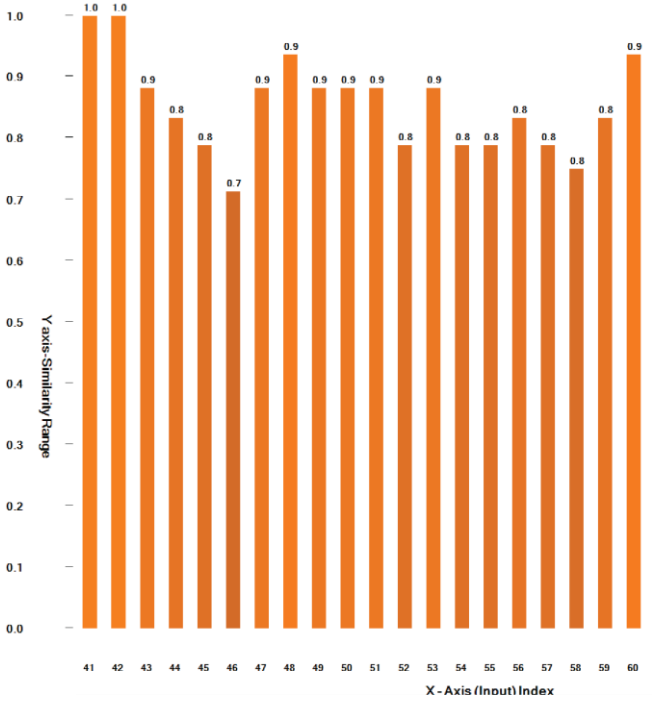

Figure 12: Similarity graph for input 41 to 60

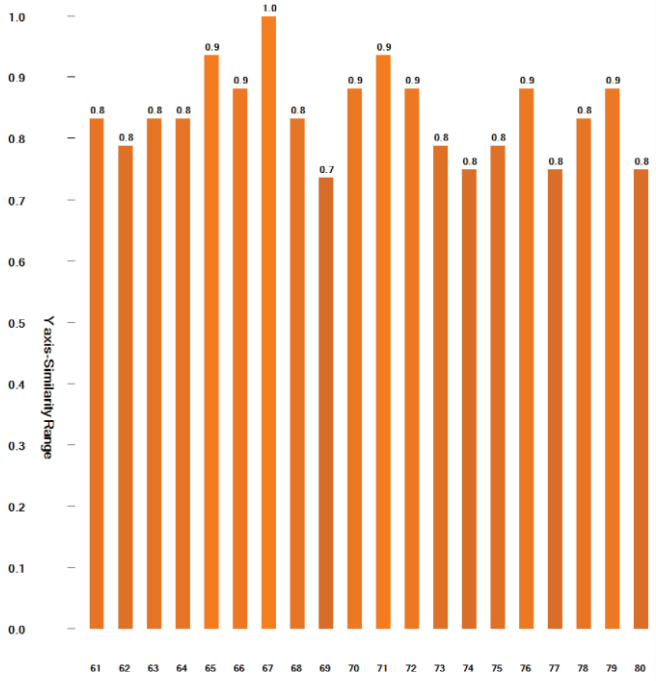

Figure 13: Similarity graph for input 61 to 80

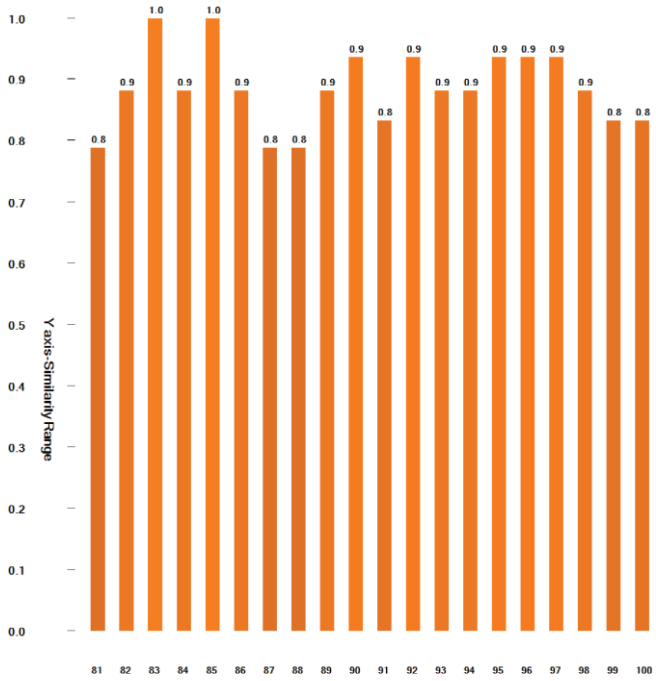

Figure 14: Similarity graph for input 81 to 100

This figure (15) represents the generated similarity graph used for output validation, but we have split it into separate parts for improved visualization.

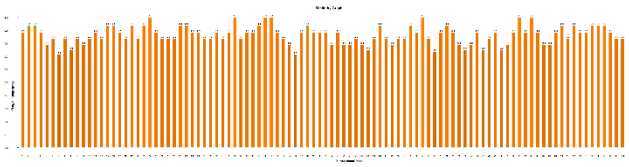

Figure 15: The similarity graphs of all numerical Inputs

Secondly, for the similarity graph in Figure (16), we used 6 images as inputs for the experiment and plotted the similarity graph between the binarized encoded inputs and the reconstructed input.

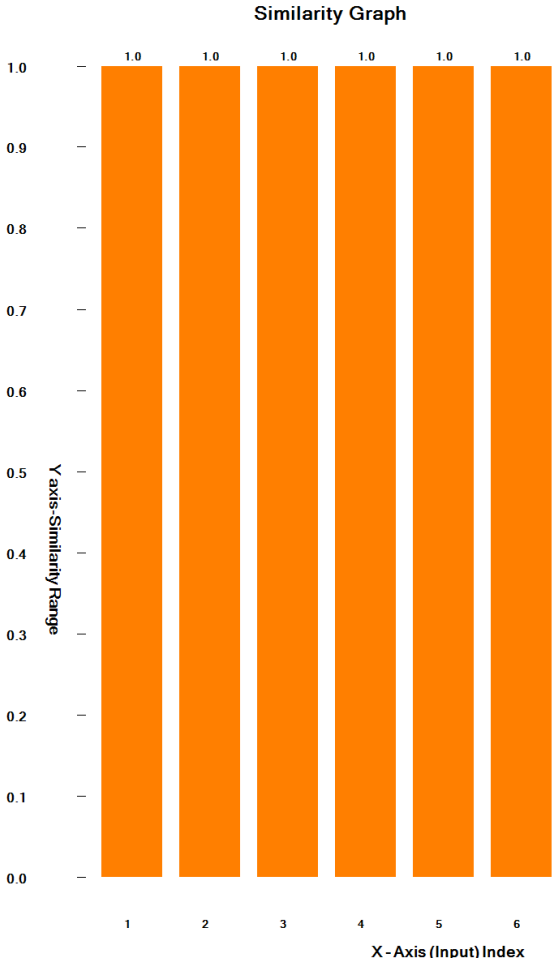

Figure 16: Similarity Graph for the Image Input.

VI. DISCUSSION

Our study focused on improving the visualization of permanence value dynamics within the HTM Spatial Pooler, emphasizing its role in input reconstruction and learning stability. By analyzing both numerical and image inputs, we explored how permanence values evolve over time and how this evolution impacts the accuracy and robustness of input reconstruction.

One of the key observations was that the permanence distributions differed significantly between numerical and image inputs. For numerical data, the permanence values exhibited a more structured pattern, closely following the encoding scheme of the input values. In contrast, for image inputs, the permanence values showed a more distributed pattern due to the complex spatial dependencies inherent in image structures. Despite these differences, our heatmap visualizations effectively captured the permanence changes for both data types, providing a clear representation of synaptic adaptation over time.

Additionally, the similarity graphs for both input types offered valuable insights into how well the reconstructed outputs preserved the original input characteristics. While numerical inputs exhibited higher reconstruction accuracy due to the more predictable encoding scheme, image inputs demonstrated greater variability, influenced by factors such as contrast, noise, and spatial complexity. These findings highlight the importance of encoding strategies in HTM-based learning models and suggest potential optimizations for improving reconstruction accuracy, particularly for complex data types like images.

Another critical aspect of our study was the role of noise in permanence evolution. We observed that small perturbations in input data led to gradual shifts in permanence values, influencing the stability of SDRs. This behavior underscores the resilience of the HTM framework in handling variations while maintaining meaningful representations. However, excessive noise introduced instability, suggesting a need for fine-tuning parameters like synaptic thresholds to optimize learning behavior.

Overall, our research contributes to a better understanding of how permanence values evolve in HTM networks and how visualization techniques can aid in interpreting these changes. Future work could explore additional data modalities, optimize encoding techniques, and enhance visualization methods to further improve interpretability and performance in real-world applications.

VII. Conclusion

In this research, we presented an enhanced visualization approach to analyze permanence value dynamics within the HTM Spatial Pooler. By leveraging the Reconstruct() method of the Neocortex API, we tracked the evolution of permanence values over time and provided meaningful visual interpretations. Our study incorporated two types of inputs—numerical values and image data—to evaluate the effectiveness of the HTM Spatial Pooler across different modalities.

Our findings demonstrated that permanence value evolution can be effectively represented using heatmaps for both numerical and image inputs, offering a clear and intuitive way to interpret synaptic stability and learning dynamics. Additionally, we introduced similarity graphs to quantify and visualize the relationship between reconstructed and original inputs, further enhancing the interpretability of HTM's reconstruction process. These visualizations provided valuable insights into how permanence distributions adapt based on input variations and noise, improving our understanding of HTM's learning behavior.

By integrating advanced visualization techniques, our approach contributes to a deeper understanding of HTM networks, particularly in terms of spatial encoding, synaptic permanence, and reconstruction accuracy. This research lays the foundation for future studies on optimizing HTM-based learning systems and exploring their applications.

VIII. References

[1] J. H. D. George, "A Hierarchical Bayesian Model of the Visual Cortex," in *IEEE International Joint Conference on Neural Networks*, 2005.

[2] S. A. D. D. J. Hawkins, "Hierarchical Temporal Memory Including HTM Cortical Learning Algorithms," 2011.

[3] J. H. S. Ahmad, "Properties of Sparse Distributed Representations and Their Application to Hierarchical Temporal Memory," *Frontiers in Computational Neuroscience*, vol. 9, no. 35, pp. 1-14, 2015.

[4] S. Pedersen, "Sparse Distributed Representations in Neural Networks," 2020.

[5] S. A. J. H. Y. Cui, "Continuous Online Sequence Learning with an Unsupervised Neural Network Model," *Neural Computation*, vol. 28, no. 11, pp. 2474-2504, 2016.

[6] A. S. J. H. M. Yavuz, "A Framework for Intelligence and Cortical Function Based on Grid Cells in the Neocortex," *Frontiers in Neural Circuits*, vol. 12, no. 10, pp. 1-22, 2018.

[7] T. R. J. L. M. Lewis, "Cortical Mini-columns and HTM: A Comparative Study," *Cognitive Science Journal*, vol. 45, no. 3, pp. 553-568, 2020.

[8] G. K. R. T. P. Mhatre, "Modeling Grid Cells with HTM: A Sparse Representation Approach to Spatial Mapping," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1345-1352, 2021.

[9] A. G. L. P. R. Vetrivel, "HTM for Adaptive Anomaly Detection in Streaming Data," *IEEE Access*, vol. 9, pp. 123456-123469, 2023.

[10] C. Z. S. A. M. Dubey, "Boosting Strategies in HTM Spatial Pooler: Enhancing Learning and Generalization," *Neural Processing Letters*, vol. 55, no. 4, pp. 1321-1345, 2022.